n8n & RAG

Intelligente Automatisierung mit Retrieval-Augmented Generation

Agenda

1. Was ist n8n?
2. Was ist RAG?
3. Wie funktioniert RAG technisch?
4. n8n + RAG – die Kombination
5. Anwendungsfälle
6. Demo-Workflow
7. Vorteile & Grenzen
8. Fazit & Ausblick

Was ist n8n?

> **n8n** ist eine quelloffene, selbst hostbare Workflow-Automatisierungsplattform – das Open-Source-Pendant zu Zapier oder Make.

Kernmerkmale

- 🔗 **400+ Integrationen** – APIs, Datenbanken, KI-Dienste
- 🧩 **Low-Code / No-Code** – visueller Workflow-Editor
- 🏠 **Self-hosted oder Cloud** – volle Datenkontrolle
- 🤖 **AI-native** – eingebaute LLM- und Vektorknoten
- 🔓 **Fair-Code-Lizenz** – kostenlos für eigene Projekte

Typische Einsatzbereiche

Bereich	Beispiel

Was ist RAG?

> **RAG** *(Retrieval-Augmented Generation)* ist eine Technik, bei der ein Large Language Model (LLM) mit relevantem Kontext aus einer externen Wissensbasis angereichert wird, bevor es eine Antwort generiert.

Das Problem ohne RAG

```
Nutzer: "Was steht in unserem internen Handbuch über Urlaubsregelungen?"
LLM:    "Ich habe keinen Zugriff auf Ihr internes Handbuch." ❌
```

Die Lösung mit RAG

```
Nutzer: "Was steht in unserem internen Handbuch über Urlaubsregelungen?"
RAG:    [sucht relevante Abschnitte] → gibt LLM als Kontext mit
LLM:    "Laut Ihrem Handbuch Abschnitt 4.2 gilt..." ✅
```

Wie funktioniert RAG technisch?

Phase 1: Indexierung (einmalig)

```
Dokumente
    │
    ▼
[Chunking]          → Text in kleine Abschnitte aufteilen
    │
    ▼
[Embedding Model]   → Abschnitte als Vektoren kodieren
    │
    ▼
[Vektordatenbank]   → Vektoren speichern (Pinecone, Qdrant, Weaviate...)
```

Phase 2: Abfrage (zur Laufzeit)

```
Nutzerfrage
    │
    ▼
```

n8n + RAG – die Kombination

n8n stellt alle notwendigen Knoten bereit, um vollständige RAG-Pipelines **ohne eigene Serverinfrastruktur** zu bauen.

Relevante n8n-Knoten

Knoten	Funktion
AI Agent	Orchestriert LLM + Tools
Embeddings	OpenAI, Cohere, Ollama u. a.
Vector Store	Pinecone, Qdrant, Supabase, In-Memory
Document Loader	PDF, Website, Google Drive, Notion
Text Splitter	Rekursiv, Token-basiert
Chat Memory	Gesprächskontext speichern

Anwendungsfälle

📄 Dokumenten-Q&A

Mitarbeiter können natürlichsprachliche Fragen an interne PDFs, Wikis oder Handbücher stellen.

> *"Welche Schritte sind für den Onboarding-Prozess vorgesehen?"*

🛒 E-Commerce Support-Bot

Produktkatalog als Wissensbasis – der Bot beantwortet Fragen zu Artikeln, Verfügbarkeit und Lieferzeiten in Echtzeit.

📊 Datenanalyse-Assistent

Reports und Rohdaten werden vektorisiert. Führungskräfte stellen Fragen wie:

> *"Welche Region hatte im Q3 den höchsten Umsatzrückgang?"*

🏥 Medizinisches Wissensmanagement

Klinische Leitlinien und Studien als RAG-Basis – Ärzte erhalten schnell kontextualisierte Informationen.

Demo-Workflow in n8n

```
[Webhook / Chat Trigger]
        |
        ▼
[Frage des Nutzers empfangen]
        |
        ▼
[Embedding erstellen]          ← OpenAI / Ollama
        |
        ▼
[Vektorsuche in Qdrant]        ← Top-K relevante Chunks abrufen
        |
        ▼
[Prompt zusammenstellen]       ← System-Prompt + Kontext + Frage
        |
        ▼
[LLM aufrufen]                 ← GPT-4o / Claude / Llama 3
        |
        ▼
[Antwort zurückgeben]          ← Via Webhook-Response / Chat
```

Vorteile & Grenzen

✅ Vorteile

Vorteil	Beschreibung
Aktualität	LLM greift auf aktuelle Daten zu
Transparenz	Quellenangaben möglich
Kostenkontrolle	Kein Fine-Tuning nötig
Datenschutz	Daten bleiben im eigenen System
Flexibilität	Einfacher Wechsel des LLMs

⚠️ Grenzen

Grenze	Erläuterung
Chunk-Qualität	Schlechtes Splitting → schlechte Antworten

Best Practices

📐 Chunking-Strategie

- Chunk-Größe: **500–1000 Token** (je nach Dokument)
- Überlappung: **10–20%** für Kontextkontinuität
- Semantisches Chunking bevorzugen

🔍 Retrieval-Qualität verbessern

- **Hybrid Search**: Volltext + Vektor kombinieren
- **Re-Ranking**: Gefundene Chunks neu bewerten
- **HyDE**: Hypothetische Antwort als Such-Query nutzen

🛡️ Sicherheit & Governance

- Dokumente mit Metadaten versehen (Abteilung, Datum, Freigabe)

Fazit & Ausblick

Zusammenfassung

> **n8n + RAG** ermöglicht es, leistungsfähige KI-Anwendungen schnell zu bauen – ohne tiefes ML-Wissen, ohne eigene Serverinfrastruktur, mit voller Datenkontrolle.

Ausblick: Agentic RAG

```
Einfaches RAG        →   Agentic RAG
(statische Suche)        (LLM entscheidet selbst, was zu suchen ist)
```

- **Multi-Step Retrieval**: Mehrstufige Suchanfragen
- **Tool Use**: LLM kann Suche, Berechnungen, APIs kombinieren
- **Self-Correction**: Antwort bewerten und Suche verfeinern

n8n unterstützt bereits heute **AI Agents** mit Tool-Calling – die Grundlage für Agentic RAG.

Ressourcen

Ressource	Link
n8n Dokumentation	docs.n8n.io
n8n AI-Knoten	docs.n8n.io/advanced-ai
n8n Community Templates	n8n.io/workflows
RAG-Konzepte (LangChain)	python.langchain.com/docs/concepts/rag
Qdrant (Vektordatenbank)	qdrant.tech
Ollama (lokale LLMs)	ollama.com